[MID-PROJECT REVIEW] EMBEDDED SYSTEMS WORKSHOP

FieldMind on the Edge

Scheduling multi-agent LLM inference across the NPU and CPU of a Snapdragon 8 Gen 3 (SM8650) QIDK

01 **Edge AI for industrial machines**

An advisor that reads a boiler's sensors and names the likely fault every 30 s, built to run on-device with no cloud.

02 **Multi-agent jobs, scheduled**

Small LLM agents queued across the Hexagon NPU and Kryo CPU, inside latency, memory and thermal limits.

TEAM · HARSHA KE ABS

Shubham Gupta
Gagan Adithya
Pranav Gangwal
Aarav Muni

TA

Aryama Murthy

PROBLEM STATEMENT

An industrial boiler turns coal into steam around the clock. When a reading starts to drift, operators have to work out why before the boiler trips.

WHAT WE MEASURE · 6 READINGS, EVERY 5 s

WATER SIDE	HEAT SIDE
Drum level	Bed temperature
Feed water flow	Drum pressure
Steam flow	Main steam temperature

WHAT GOES WRONG · FAULTS WE SIMULATE

`Feed valve seizure` `Feed-pump suction` `Tube leak` `Wet coal` `Feeder trip` `High-CV coal` `Fouling drift`

Several faults move the six readings the same way. The context often sits in notes and lab reports, not in the sensors.

FieldMind names the likely cause every 30 s

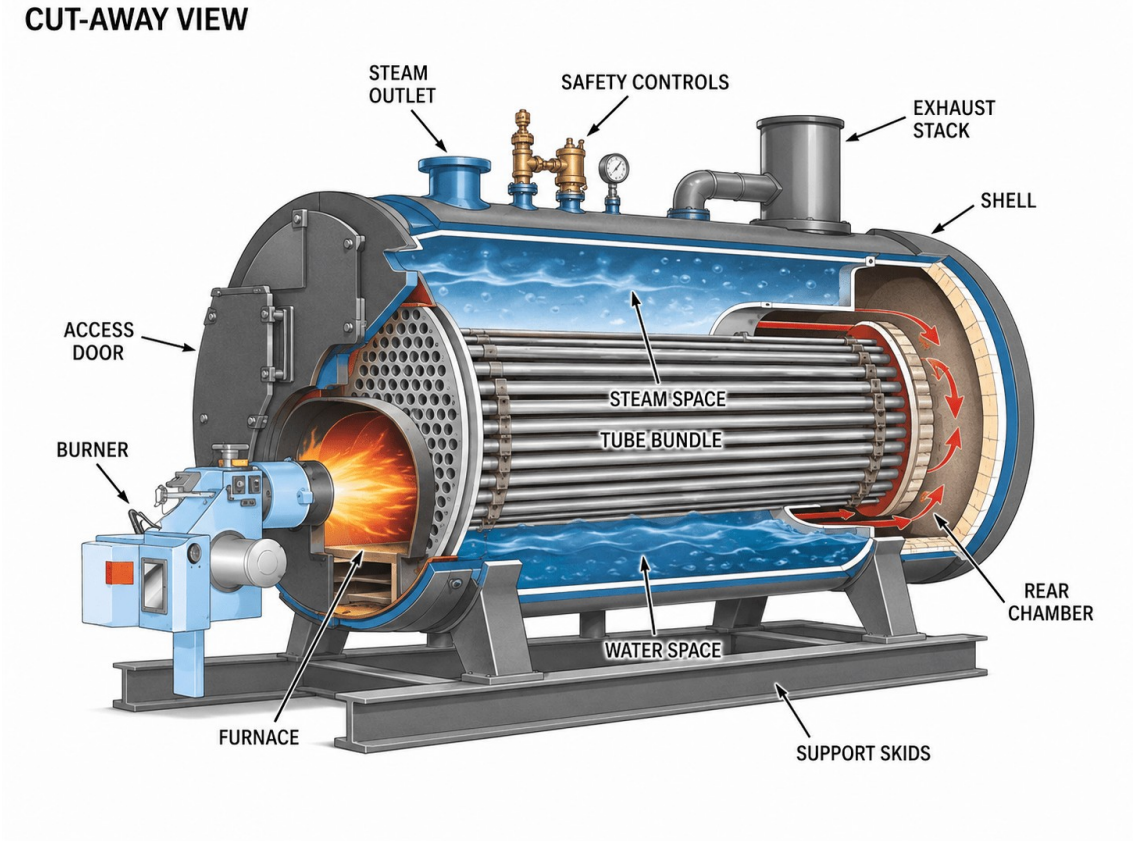


Illustration: a packaged boiler. Our simulated plant is a 67 TPH coal-fired fluidised-bed (AFBC) boiler.

Rank the likely cause, every 30 seconds

INPUTS

01 SENSOR TIME SERIES

6 boiler tags, sampled every 5 s

drum_level	feed_water_flow
steam_flow	drum_pressure
bed_temp_avg	ms_temperature

02 CASE LIBRARY

13 real failure reports

Root-cause analyses of real incidents, each with an expected sensor signature

03 ENGINEER NOTES

Timestamped notes and records

Plus coal lab reports and maintenance history

EVERY 30 s TICK

A Sensors → facts

Code turns the sensor window into text facts:
`“drum level falling 2.7 %/min”`

B Match the library

Facts are scored against each case's signature; notes are searched.

C Rank the causes

Most likely cause first, citing the facts that support it.

OUTPUT

PUBLISHED EVERY TICK

PLANT STATE

NORMAL | **DEVIATION** | ALARM | TRIP

RANKED CAUSES

1	RCA-11	Tube leak
2	RCA-01	Feed valve seizure
3	RCA-16	Feed-pump suction

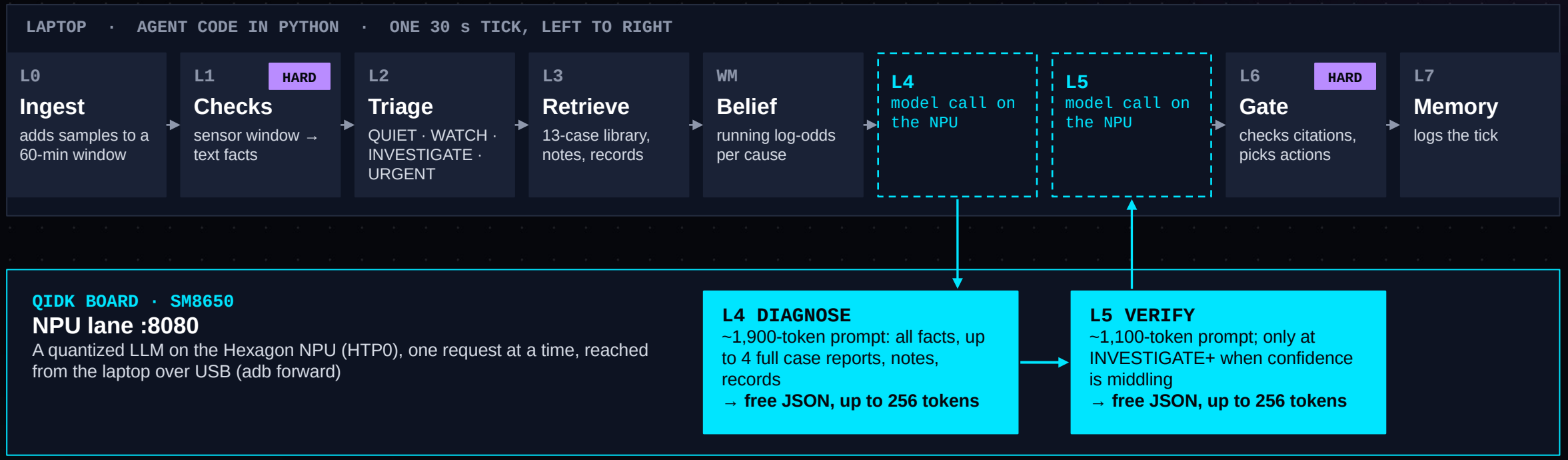
CHECKS AND ACTIONS

Picked from a fixed catalogue of 16

Illustrative output

Plant data simulated (67 TPH coal-fired AFBC boiler) · failure reports from real incidents · advisory only, never controls the plant

One agent, one big prompt, every 30 s



THE TICK WAITS

L4 and L5 run one after the other inside the tick. The next tick starts only when both return.

QUIET = NO MODEL

If triage says QUIET, no model runs at all. Model work only happens when something is off.

THE MODEL'S ORDER WINS

The model's ranking replaces belief's; belief only supplies the confidences.

TWO RULES

The model never sees raw numbers (~4,300 per tick → 150–300 tokens of facts) and never has the last word (the gate).

The benchmark, and a first run on the CPU

HOW THE DATASET IS BUILT · SIMULATED PLANT, REAL REFERENCES

01	**Physics simulator**	Integrates mass and energy balances, IAPWS-IF97 steam tables, combustion and shrink-and-swell. A fault changes a physical driver; all six tags respond.
02	**Calibrated on real data**	120 h of a real boiler (86,400 rows at 5 s). Noise and cross-tag correlation matched: steam ↔ bed 0.88 vs 0.87 real.
03	**Validation gate**	8 checks per episode: mass and energy close, noise, physical ranges. A failing episode is never written.
04	**30 episodes**	6 normal + 24 with a fault. Each has the six tags, timestamped notes, plant records and a hidden answer key.
05	**Real failure library**	13 cases from an 18-case AFBC root-cause study; 5 held out to test faults the agent has never seen.
06	**Replayed tick by tick**	The harness only shows data up to “now”, so detection lead time can be scored.

EPISODES PER FAMILY

N 6 | A 6 | B 5 | C 6 | D 4 | E 3

FIRST ON-DEVICE RUN · EARLY SEPTEMBER

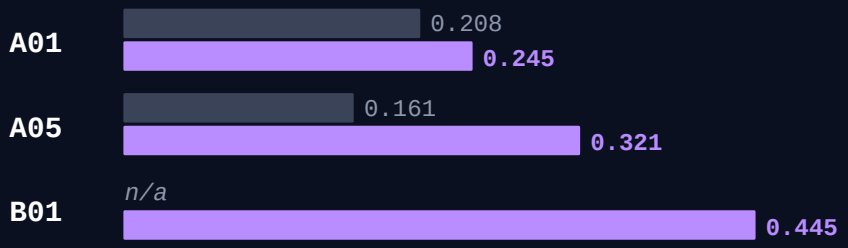
Gemma 3 1B on the Kryo CPU, LiteRT-LM runtime

45–52 s
p95 per model call, against a 30 s tick

33→88 °C
chip in one episode, into throttling

2,066
prompt tokens, against a 1,280 NPU context

TOP-1 ON SEPARABLE FAULTS — retrieval only / LiteRT, CPU



6 episodes · ~0 parse failures · 8–14% of ticks over budget

The model helped, but the CPU couldn't answer inside the tick. That's why we moved to the NPU.

Getting Llama 3.2 3B onto the Hexagon NPU

01
Start from BF16 weights

Meta's Llama 3.2 3B Instruct as released: 16 bits (BF16) for every weight.

02
Quantize to 4 bits

Weight matrices to 4-bit Q4_0, the token table to 8-bit Q8_0. One 2.01 GB file.

03
Run it with GenieX

GenieX's llama.cpp engine places the layers on the Hexagon NPU, behind one server per lane.

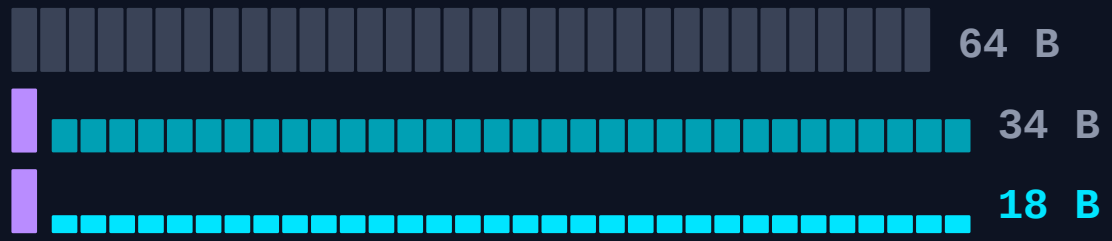
04
Prove it's on the NPU

The startup log is checked on every launch: all 28 blocks and the output layer on the NPU.

PIPELINE `convert_hf_to_gguf.py → llama-quantize --pure --token-embedding-type q8_0 --output-tensor-type q8_0 <src> <dst> Q4_0`

Converted on 3 Oct from the original BF16 weights · the quantizer comes from the same llama.cpp version (b11371) as the runtime, so the file is written and read by identical code

HOW BLOCK QUANTIZATION WORKS · ONE BLOCK OF 32 WEIGHTS



Purple = one shared 16-bit scale per block. Each weight is stored as a small integer and multiplied back by that scale: about 4.5 bits per weight at Q4_0, instead of 16.

OUR FILE VS OLLAMA'S DEFAULT Q4_0

	OURS	OLLAMA
Weight matrices	**Q4_0**	Q4_0
Token table	**Q8_0 · 8-bit**	Q6_K · 6-bit

One uniform Q4_0 + Q8_0 recipe, rebuilt the same way for all four models.

Prefill flies. Decode is the wall

LLAMA 3.2 3B ON HTP0

986 tok/s
prefill: reading the prompt

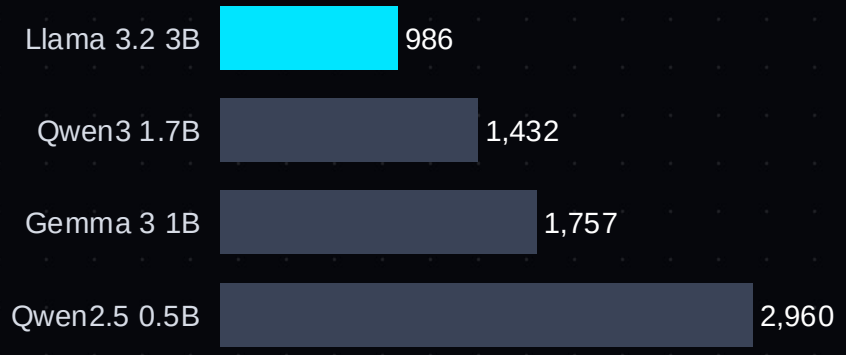
21 tok/s
decode: writing the answer

A 60-token answer costs about 2.9 s; a 700-token prompt about 0.7 s. Answer length sets the call time.

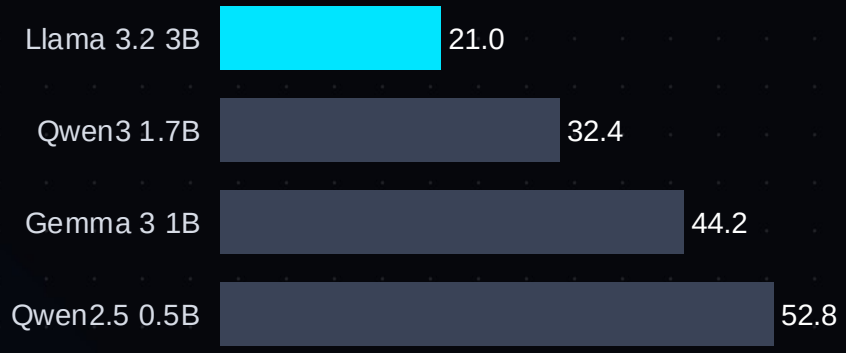
ONE DIAGNOSIS + ONE VERIFICATION

Llama 3.2 3B	**5.34 s**
Qwen3 1.7B	**3.51 s**
Gemma 3 1B	**2.63 s**
Qwen2.5 0.5B	**2.06 s**

PREFILL · tok/s



DECODE · tok/s · 40–56× SLOWER

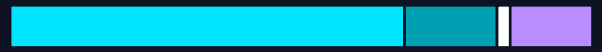


PROOF IT RUNS ON THE NPU

LAYERS ON HTP0, FROM THE STARTUP LOG

Llama 3.2 3B	**29/29**
Qwen3 1.7B	**29/29**
Gemma 3 1B	**27/27**
Qwen2.5 0.5B	**25/25**

LLAMA 3B MEMORY, MiB



1,911.90	NPU model
448.00	NPU KV cache
64.01	NPU compute
399.23	CPU token table

All 197 matrix multiplications per pass, norms, RoPE and attention on HTP0. Only the input token lookup runs on the CPU.

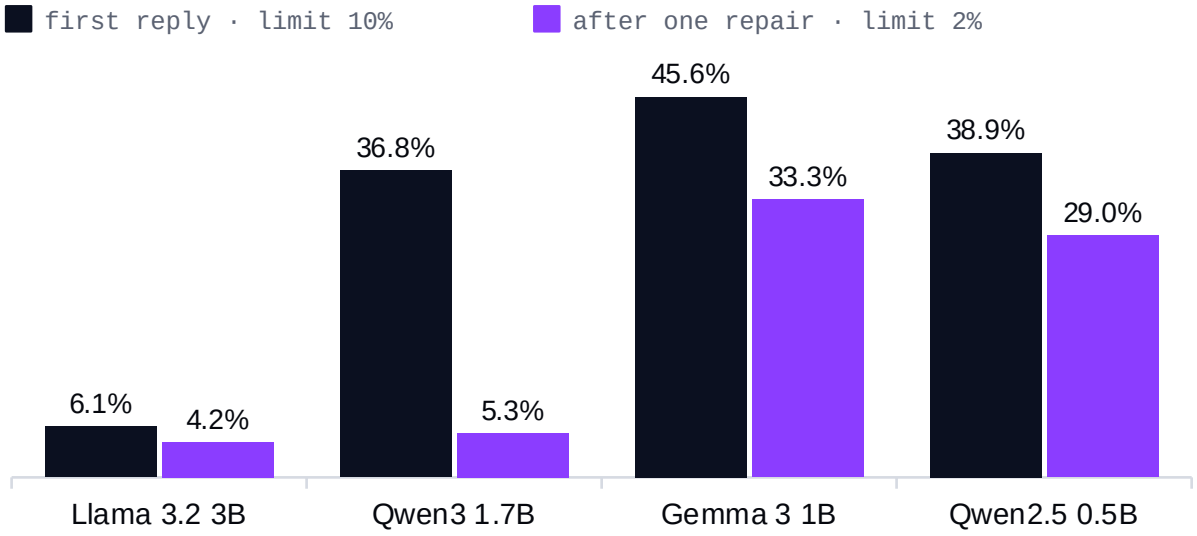
Without ADSP_LIBRARY_PATH the server silently runs on the CPU, so code checks the log on every launch.

Same 4-bit recipe for all four · 5 × 700/60 and 5 × 350/30-token calls per model · server timing counters

Small models were fast, but broke their answers

AUTOMATIC CAMPAIGN rules fixed in advance · single agent · 3 dev episodes per model · free JSON, 256-token cap · no grammar

BROKEN JSON, SHARE OF REPLIES



ROUND 1 DROP RULE **30% / 10%**
Qwen3 1.7B and Gemma 3 1B dropped after round 1. Their bars are round-1 rates; Llama and Qwen2.5 show the final round.

FINAL ROUND

	LLAMA 3B	QWEN 0.5B	CODE ONLY
Right cause group at rank 1	**0.507**	0.418	0.501
Broken JSON, first / repaired	**6.1/4.2%**	38.9/29.0%	–
Cited facts that exist	**0.828**	0.179	–
Mean diagnosis call	**11.4 s**	4.9 s	–
Answers hitting the 256 cap	**4.7%**	50.3%	–
Ticks over the 30 s budget	**16.8%**	0.25%	–

NO MODEL PASSED EVERY LIMIT
Llama 3.2 3B missed only one: 4.2% broken after repair, against 2%. 10 of its 11 failures were answers cut off at the 256-token cap. Chosen by a recorded override.

LESSON Speed isn't enough. Answers must be short and structured, which led to the answer grammar later (0 unusable answers).

Full benchmark: Llama 3.2 3B as the single agent

636
model calls: 147 · 196 · 43 · 250

81 / 500
ticks with a model call over the 30 s budget

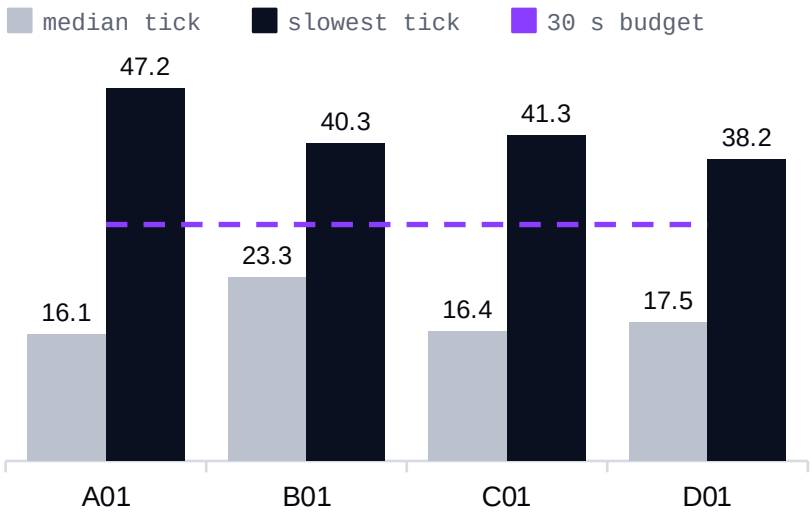
71
unusable answers, 11% of calls

1.09 M
prompt tokens in; 0.11 M answer tokens out

36.8→64.8
°C chip during A01, calls back to back

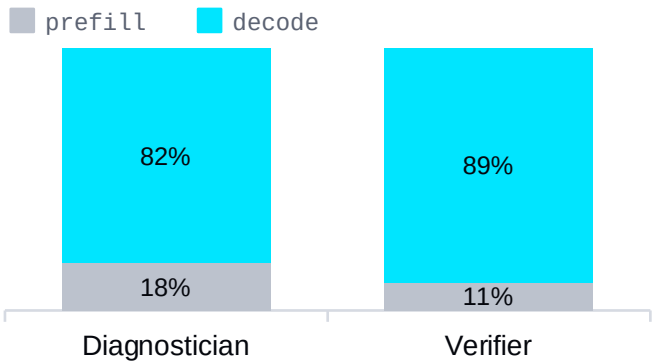
0 / 636
calls on the CPU lane, which sat idle

TICK TIME PER EPISODE, s



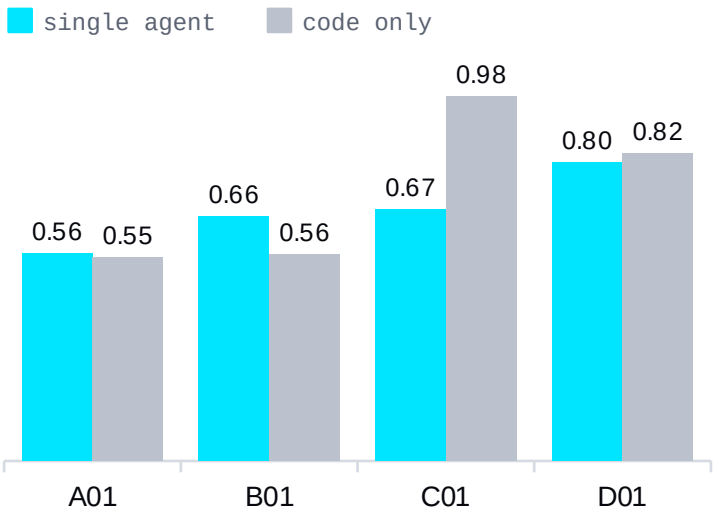
over 30 s: 19/106 · 33/128 · 5/43 · 24/223 ticks

WHERE MODEL TIME GOES



	Diag.	Verif.
calls	657	177
prompt, median	1,931	1,107
answer, mean	159	193
decode tok/s	13.6	14.4
median call	13.8 s	14.4 s

RIGHT CAUSE GROUP AT RANK 1



mean 0.672 vs 0.726 for code alone

UNUSABLE ANSWER a reply the gate could not use: free JSON that doesn't parse, or an answer cut off at the 256-token cap (14 · 34 · 3 · 20 per episode). That call's model time is wasted.

Llama 3.2 3B on the NPU · A01 7 Oct, B01–D01 5 Oct · one run each · tick times include laptop code; call times and tok/s are board numbers

Five reasons one big agent doesn't fit the chip

01	Too slow for the tick	The model blocks the 30 s tick: median tick with a model call 16–23 s, slowest 38–47 s.	81 / 500 ticks over budget
02	Decode-bound	Writing 160–190-token answers at ~14 tok/s takes most of every call; reading the prompt is the small part.	82–89% of model time decoding
03	Unreliable format	Free JSON came back broken or cut off at the 256-token cap. In A01 only 76% of cited facts existed.	71 unusable answers
04	No better than code alone	It helped on B01 but badly hurt C01 (0.674 vs 0.977). 77 ticks helped, 81 harmed.	0.672 vs 0.726 code only
05	One unit flat out, one idle	Calls run back to back, so the NPU never rests and the chip heats from ~37 to 65–70 °C. The CPU lane sits unused.	1 of 2 compute units used

SO THE NEXT DESIGN MUST

- never make the tick wait
- keep answers short
- enforce the answer format
- use the NPU and the CPU
- keep code's ranking as base

Small jobs, two lanes, a tick that never waits

LAPTOP · AGENT CODE IN PYTHON

HARD PATH · CODE ONLY, EVERY TICK, < 200 ms

Sensor agent
window → facts; each fact ID carries its tick (t84.F1)

→

Triage
QUIET means no model work at all

→

Retriever + belief
candidate cases; log-odds ranking per cause

→

Gate + memory
checks every answer, applies the merge rule, publishes

Scheduler
one queue per lane; a side is re-asked only when its evidence fingerprint changes

Blackboard
shared state; one writer per section, enforced in code; model jobs can't write

jobs ↓ ↑ answers → gate

jobs ↓ ↑ answers → gate

QIDK · NPU LANE :8080 · HTP0 · LLAMA 3.2 3B

Water diagnostician
ranks causes from drum level, feed flow and steam flow

Heat diagnostician
ranks causes from bed temp, drum pressure and steam temp

~750-token prompt → line-number answer, ≤ 60 tokens

QIDK · CPU LANE :8081 · 6 THREADS · GEMMA 3 1B

Verifier
passes or fails each of the top 3 claims; can only lower confidence

Note reader
codes engineer notes into fixed words (v2; see slide 15)

~400-token prompt → short answer, ≤ 60 tokens

ONE REQUEST AT A TIME

Each lane is a single-slot server, so the scheduler keeps one queue per lane.

ASK ONLY ON CHANGE

Unchanged evidence reuses the last checked answer: diagnosis calls halved on the dev set (2,005 → 1,049).

LOCKSTEP BENCHMARKS

Every measured run waits for its own jobs, so its numbers compare fairly with the single agent.

Every weakness gets a design answer

	SINGLE AGENT	MULTI-AGENT DESIGN	WHY IT HELPS THE CHIP
01	The tick waits 16–23 s for the model	**A code-only hard path publishes every tick within 200 ms; model work is queued as jobs**	The deadline no longer depends on NPU latency
02	One ~1,900-token prompt and a ~160-token free-JSON answer	**~750-token prompts split by plant side; answers capped at 60 tokens, written as line numbers** Line numbers: the model points at numbered prompt lines; code maps them back to case and fact IDs.	Decode is the bottleneck, so short answers cut call time most
03	Free JSON breaks: 71 unusable answers	**A GBNF grammar built per request from the lines shown; its longest answer fits the cap** GBNF: a grammar the server enforces token by token, so only valid answers can be generated.	Every NPU call produces a usable answer
04	One model on one unit; the CPU idle	**Two lanes, two models: diagnosis on the NPU, verifier and note reader on the CPU**	The CPU checks tick t while the NPU already diagnoses tick t+1
05	A diagnosis on every non-quiet tick	**A side is re-asked only when its evidence fingerprint changes**	Fewer NPU calls: halved on the dev set

WHERE EACH JOB RUNS

Gemma 3 1B, measured on both units

PREFILL · tok/s

1,757 NPU **341–357** CPU

About 4–5× faster on the NPU: long prompts go there.

DECODE · tok/s

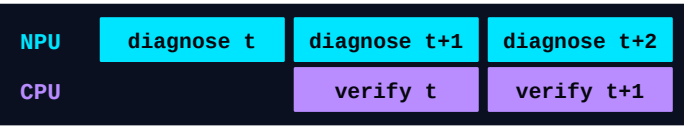
44.2 NPU **37.0** CPU

Close: a small model writing short answers keeps up on the CPU (58.4 vs 44.2 in one comparable call).

NPU ← diagnosticians, ~750-token prompts
CPU ← verifier, ~400-token prompts, ~25-token answers

NPU: screening, 10 calls. CPU: 280 + 174 real calls. Prompt size, grammar and chip temperature differ.

PIPELINED ACROSS TICKS
the two lanes overlap



While the CPU verifies tick t, the NPU is already diagnosing tick t+1, so checking adds no time on the NPU's critical path.

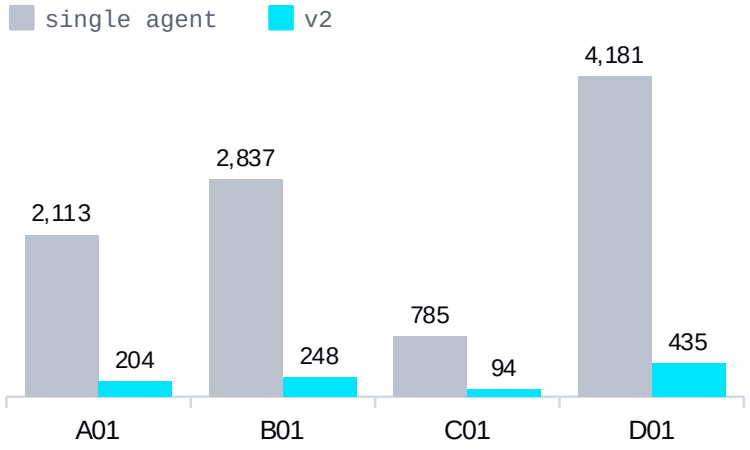
ALSO BUILT **Earliest-predicted-finish placement:** finish = lane free + prompt ÷ prefill rate + answer cap ÷ decode rate. Implemented, not yet evaluated.

10× faster, zero broken answers, accuracy fell

WALL TIME, 4 EPISODES	MEDIAN MODEL CALL	MEDIAN TICK WITH A CALL	TICKS OVER 30 s	UNUSABLE ANSWERS	PROMPT TOKENS IN
9,915 s → **980 s**	14.8 s → **2.9 s**	18.3 s → **2.3 s**	81 → **0**	71 → **0**	1.09 M → **0.22 M**

single agent → multi-agent v2 · same 4 fault episodes · merge rule: the model's order wins (as in the single agent)

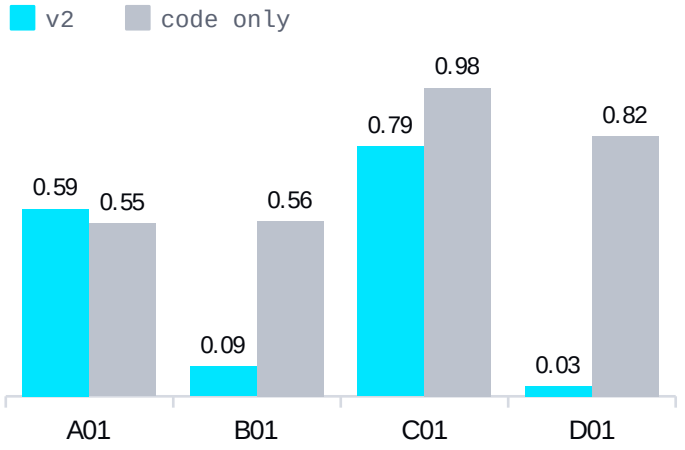
WALL TIME PER EPISODE, s



THE TWO LANES IN THE MULTI-AGENT RUNS

	NPU diag.	CPU verif.	CPU notes
calls	1,492	280	174
prompt, median	749	406	404
answer, mean	34	25	17
prefill tok/s	809	341	357
decode tok/s	15.9	37.0	27.1
share decoding	70%	36%	36%
median call	3.1 s	1.9 s	1.8 s

RIGHT CAUSE GROUP AT RANK 1



FORMAT FIXED FIRST · ANSWER GRAMMAR
First runs: 26–42% of diagnoses and 68–100% of Gemma answers unusable. With the grammar: A01 26/84 → 0/57, B01 47/112 → 0/52, then 0 in every run. No cost on the NPU (20.2 tok/s with or without); on the CPU it slowed Gemma's decode (62 → 46 tok/s, one call).

THE COLLAPSE **mean 0.378 vs 0.726 code only**
single agent 0.672 · 57 ticks helped, 294 harmed · first right 50.3 min after onset (single 11.0)

Belief, the model, and the published answer

BELIEF · CODE ONLY

A log-odds score per cause updated every tick from how well the facts match each case's sensor signature

MODEL · ON THE NPU

Ranks belief's top candidates up to 4, citing facts by line number; the gate checks it, the verifier can only lower confidence

MERGE RULE
decides how far the model may move belief's order

PUBLISHED RANKING

What the operator sees each tick. No rule ever writes into belief, and the plant state and alarms always come from code, so a model failure can never hide an alarm.

WHAT IT CHANGED FOR V3 The model is right where belief is weak (0.88 vs 0.18) and wrong where belief is sure (0.16 vs 0.79). So hybrid lets it lead only when belief is weak or tied, and never with a filler case.

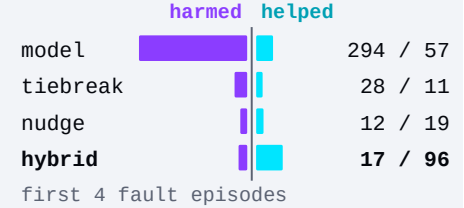
VERSION	RULE	HOW THE PUBLISHED TOP CAUSE IS CHOSEN	RANK-1
single, v2	model	The model's order wins outright; belief only supplies the confidence numbers.	0.378
v2 fix	tiebreak	Belief's order is published; the model may only reorder causes within 0.35 log-odds of belief's leader.	0.688
v2 fix	nudge	Belief's order, but each fresh answer adds +0.35 to the model's top cause and +0.175 to its second, capped at 1.2 per cause per episode.	0.750
v3	**hybrid**	Belief weak (best score < 0) or tied (top two within 0.35): the model's ranking leads. Otherwise nudge. Guard: never lead with or boost a sensor-flat “filler” case.	**0.836**

rank-1 = right cause group published on top · first 4 fault episodes · code alone 0.726

WHO IS RIGHT, BY BELIEF'S STATE · ALL 10 FAULT EPISODES

belief's state	ticks	belief right	nudge right	model pick
clear leader ≥ 0.35	521	**0.79**	0.85	0.16
near-tie < 0.35	528	0.32	0.34	**0.41**
weak: best < 0	60	0.18	0.45	**0.88**
a filler case leads	86	0.00	0.01	0.21

TICKS VS BELIEF ALONE



Four problems found in the model's own picks

01 The model overwrote a better code ranking

EVIDENCE D01, tick 121: belief had the true cause at 0.98; the model put a fault that is flat on every sensor first, and it was published.

FIX Merge rules that keep belief's order as the base: tiebreak (reorder near-ties only), then nudge (capped boosts).

RANK-1 ACCURACY · FIRST 4		
model wins		**0.378**
tiebreak		**0.688**
nudge		**0.750**
harmed ticks	294	**12**

02 A shortcut hidden in the prompt

EVIDENCE Each case line carried a look-alike group letter. In 89% of 515 answers the model wrote the most common letter in the list; on B and D it did worse than a random pick.

FIX Remove the group letters from the prompt and the answer.

MODEL'S OWN FIRST PICK		
all answers	0.33	**0.65**
B tube leaks	0.04	**0.70**
D high-CV coal	0.02	**0.70**
filler first	39%	**16%**

03 The model is strong exactly where code is weak

EVIDENCE Split by belief's own state over ten episodes: when belief is unsure the model is far better; when belief has a clear leader the model is far worse.

FIX hybrid: the model leads when belief is weak or tied, otherwise capped nudges; a guard blocks sensor-flat 'filler' causes.

RIGHT · BELIEF vs MODEL		
weak	0.18	**0.88**
near-tie	0.32	**0.41**
clear leader	0.79	**0.16**

04 The CPU note reader invented facts

EVIDENCE Forced by the grammar to code every note, Gemma turned 'fire extinguisher refill due' into FEED_VALVE DOWN.

FIX No reader: the diagnostician reads the raw note inside a data fence. The CPU lane now runs only the verifier.

NPU PROMPT · BEFORE → AFTER		
median tokens	748	**731**
max tokens	914	**873**
median call	3.01 s	**3.05 s**

RIGHT CAUSE GROUP AT RANK 1 · FIRST 4 EPISODES

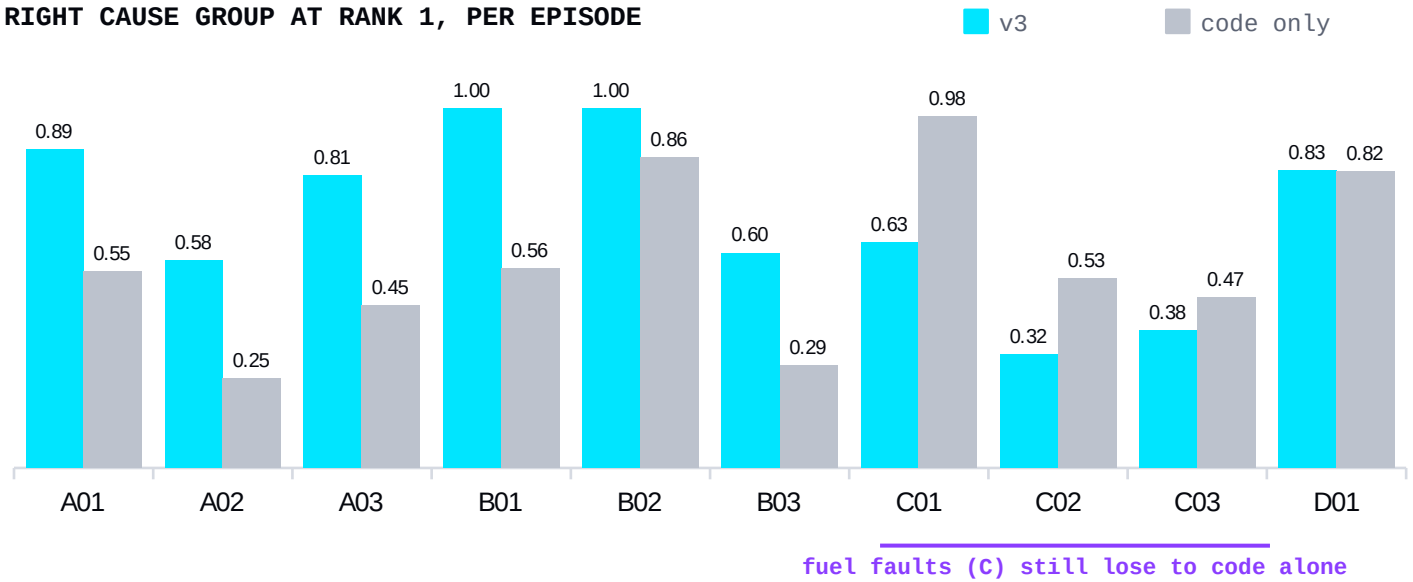
0.672	0.378	0.688	0.750	0.836	0.726
single	v2 model	tiebreak	nudge	v3 hybrid	code only

v3: as fast as v2, and ahead of code alone

RIGHT GROUP AT RANK 1	RIGHT GROUP IN TOP 3	RIGHT BEFORE THE TRIP	FIRST RIGHT AFTER ONSET	TICKS HELPED / HARMED
0.703	**0.908**	**0.76**	**9.6 min**	**224 / 38**
0.575 code only	0.722 code only	0.41 code only	14.9 min code only	vs code only

all 10 fault episodes · same board, lanes and flags as v2 · prompt v3: no group letters, raw notes · merge rule hybrid

RIGHT CAUSE GROUP AT RANK 1, PER EPISODE



STILL CHEAP ON THE CHIP

first 4 episodes, single agent → v3

wall time	9,915 s	**821 s**
median model call	14.8 s	**3.1 s**
median tick with a call	18.3 s	**1.3 s**
ticks over 30 s	81	**0**
model calls	636	**254**
prompt tokens in	1.09 M	**0.18 M**

NPU busy ~1.5–1.7 s per tick with a call, out of 30 s: most of the NPU's time is free headroom.

Lead time before the trip: A01 22.2 min (single 17.2) · D01 114.8 min (single 108.3)

HONEST LIMITS C: a look-alike pair belief always scores as a tie, so hybrid lets the model lead (cool re-run within 0.04) · v3 tuned on these episodes, one run each; dev-set check pending · top cause flips 8.2× per 100 ticks (nudge 4.7)

A model that picks, not writes, on the NPU

ON THE BOARD Llama writes its answer one token at a time. **A decision model reads the question and options once, then scores each one.**

01
Choose the model

bev-decider 0.4B: the first 20 layers of Qwen3-0.6B plus a small scoring layer. Trained on general choices, not boilers.

02
Shrink it to 8 bits

Every weight matrix stored as 8-bit Q8_0: a 506 MB file, plus a 29 MB scoring layer kept at full precision.

03
Write our own server

bev-decide, a small C++ server. It reuses the llama.cpp build already on the board (b11371), so nothing new was installed.

04
Prove it's on the NPU

The startup log shows all 21 of 21 layers on HTP0. Every file is checksum-checked before the server starts.

HOW ONE DECISION WORKS · NO TEXT IS WRITTEN

1 Read once

up to 9 plant facts + the question: “Which cause best explains these facts?”

→

2 Read each option

- RCA-11 · tube leak
- RCA-01 · feed valve seizure
- RCA-16 · feed-pump suction

→

3 Score them



a probability per option

The question is read once and every option is read on top of it, so one more option costs only that option's few tokens.

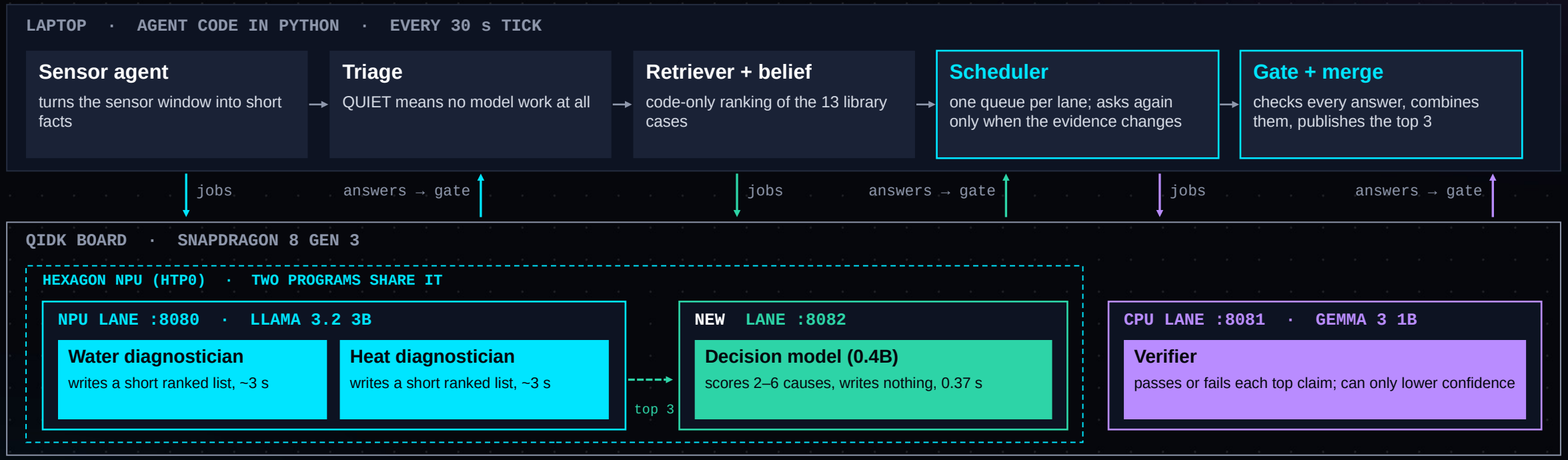
MEMORY ON THE NPU (HTP0)



Model weights	476.6 MiB
Working memory (KV cache)	320.0 MiB
Scratch space for the maths	298.8 MiB

≈ 1.07 GiB in all. It runs as a second program on the NPU, next to Llama's server.

A third lane that decides instead of writing



switch the decision model off and the system gives exactly v3's answers (20 test runs, 3,440 ticks, 0 differences), so any change in the results is the decision model's

ONLY WHEN THINGS CHANGE

Asked again only when its facts or options change: 633 calls on 1,053 ticks with something going on.

WHAT IT SEES

Up to 9 short facts (~400 tokens), one fixed question, and 2–6 causes: belief's top 3 plus the diagnosticians' top 3.

HOW MUCH SAY IT GETS

The same small boost as Llama (+0.35 top pick, +0.175 second), sharing one limit of 1.2 per cause.

SAFETY CHECKS

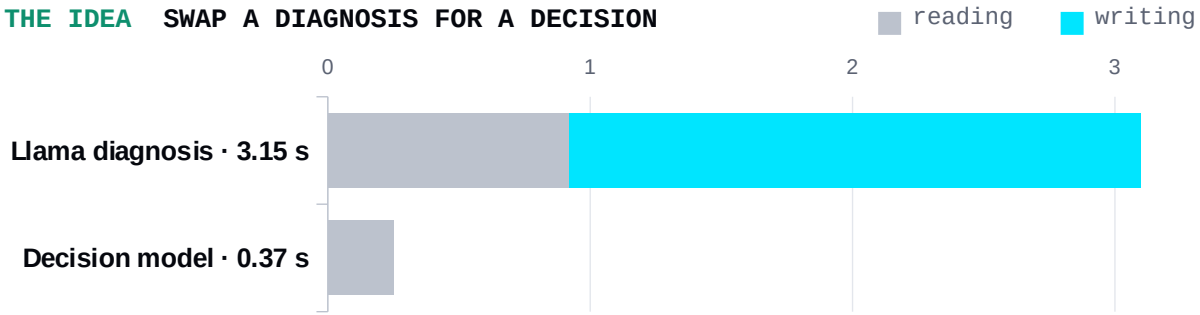
All 633 answers passed the gate. It can never push a cause that no sensor shows, and never edits belief.

Replace the diagnostician: 8× faster per answer

ONE DECISION	TOKENS WRITTEN	READING SPEED, NPU	NO SLOWDOWN TO LLAMA	FAILED ANSWERS
0.37 s	**0**	**1,607 tok/s**	**2.98 s**	**0 / 633**
3.15 s per Llama diagnosis	34 per Llama diagnosis	785–843 tok/s for Llama 3B	3.05 s without it	0 ticks over 30 s

goal: prove a decision model runs on the QIDK beside Llama · 12 episodes · call times measured on the board

THE IDEA **SWAP A DIAGNOSIS FOR A DECISION**



IF IT REPLACED THE DIAGNOSTICIANS · PROJECTION, NOT YET RUN

DIAGNOSIS TIME, 12 EPISODES	TOKENS THE DIAGNOSTICIANS WRITE
1,605 s → ≈ 188 s	17,284 → 0

509 diagnosis calls × 0.37 s per decision, using the times measured above

TOKENS · V3 VS BEV-DECIDER-3

all 12 episodes, counted on the board

	v3	bev-dec-3
Llama diagnostician		
calls	509	**509**
tokens in, per call	722	**722**
tokens out, all calls	17,284	**17,360**
Decision model		
calls	–	**633**
tokens in, per call	–	**398**
tokens out, all calls	–	**0**
All models		
tokens written	19,785	**19,803**
model time	1,794 s	**1,981 s**

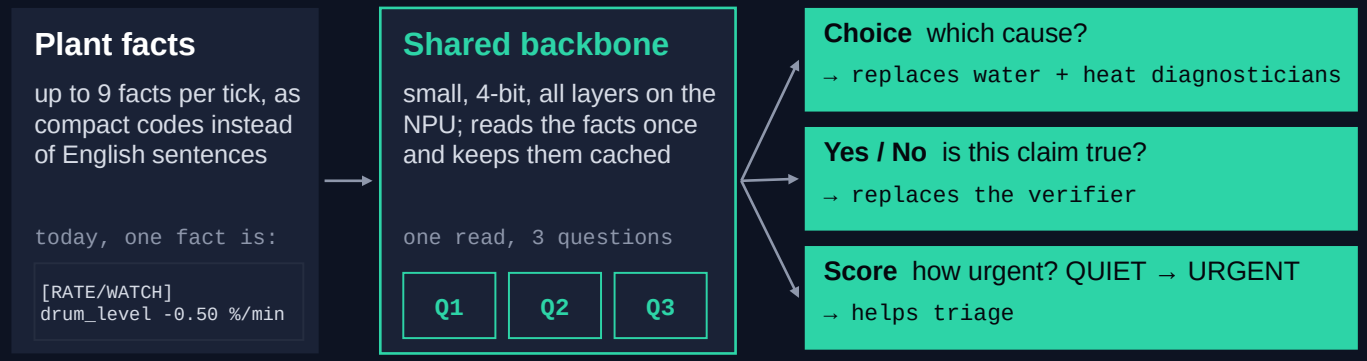
WHY THE TOTAL ROSE **In this proof of concept it ran alongside Llama: 252k extra tokens read, none written, +187 s. Swapping it in for the diagnostician removes the writing.**

LIMITS borrowed model, not trained on boilers, so accuracy was not the goal (0.680 vs v3's 0.703) · board version not yet checked against the original · one run per episode · energy not measured

Next: a decision model built for the chip

NEXT STEP bev-decider proved the idea runs on the QIDK. **Next we design our own, from the ground up, for FieldMind's questions and the NPU.**

ONE MODEL · READ THE FACTS ONCE, ASK SEVERAL QUESTIONS



The facts are read once and every question reuses them, the same trick bev-decide already uses for options. No answer is ever written out.

BORROWED TODAY VS BUILT FOR FIELDMIND

	bev-decider	ours (goal)
Trained on	support tickets, general choices	**our simulator's boiler episodes**
Size	0.4B, 20 Qwen3 layers	**smaller, sized for the NPU**
Numbers	8-bit Q8_0	**4-bit Q4_0, like Llama**
Input	English facts, ~400 tokens	**compact codes, fewer tokens**
Questions	1: pick a cause	**3: choice, yes / no, score**
Its vote	shares Llama's boost limit	**its own limit**

HOW WE'LL KNOW IT WORKS · TO BE MEASURED

AS ACCURATE AS V3

At least v3's 0.703 on the same 10 fault episodes, checked family by family.

FASTER ON THE CHIP

Time per decision and NPU memory measured on the board, against today's 0.37 s and 1.07 GiB.

REPLACES, NOT ADDS

Benchmark with the diagnosticians switched off, so the writing time really disappears.

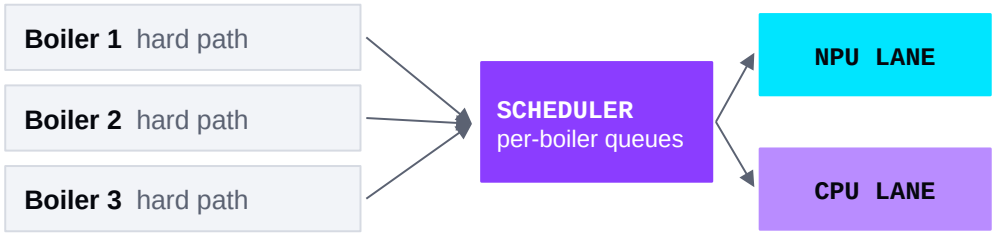
CHECKED FIRST

Compare the board version with the original model before every benchmark.

One chip, many boilers; many chips, one operator

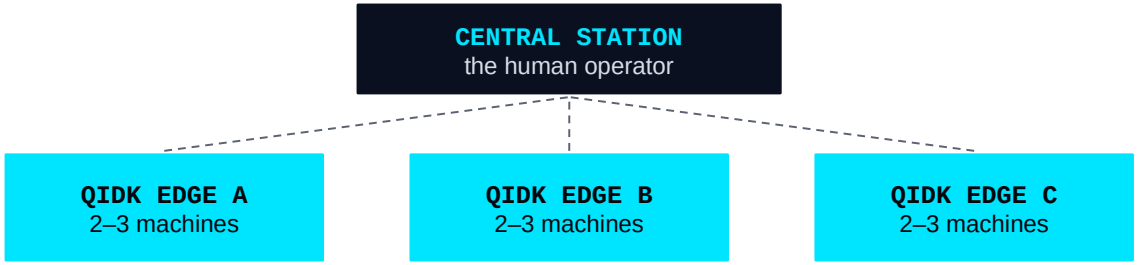
~1.5–1.7 s	1.3 s	448 MiB	11.1 GiB
NPU work per 30 s tick (v3)	median tick with a model call (v3, lockstep)	KV cache per 4,096-token context (3B)	board memory, shared by all servers

01 Many boilers on one chip



Shared lanes	every boiler's hard path feeds the same two model lanes; the NPU is idle most of each tick today
Priority by danger	a boiler near a trip jumps the queue; fair sharing stops one noisy boiler starving the rest
Memory trade-off	parallel server slots each need KV cache, so it is one slot and a queue, or more slots and more memory

02 A fleet of edge devices, one operator



Data stays local	each device runs the full agent; raw sensor data never leaves it
Tiny uplink	only the published assessment goes up: state, ranked causes, actions
Alarms stay on-device	plant state is computed by code on each unit, so a lost link never hides an alarm

TO MEASURE boilers per chip before a deadline is missed, queue delay under a burst of faults, memory per extra slot. Not yet measured.

Three experiments next on the Snapdragon

01

Move the agent onto the board

TODAY

Agent code runs in Python on a laptop; only model calls cross USB to the QIDK.

So tick times are laptop times; the code path takes 2–90 ms per tick against a 200 ms budget.

2–90 ms code path per tick, on the laptop

NEXT

Run the agent on the Kryo CPU next to the model servers: the CPU lane uses 6 of the 8 cores, leaving 2 for agent code.

WE'D MEASURE

true on-device tick time · contention with the CPU lane · no USB in the loop

02

Try other quantization formats

TODAY

Every model is pure Q4_0 with a Q8_0 token table: one uniform recipe.

The pinned Hexagon backend also accepts Q4_1, IQ4_NL, MXFP4 and the K-quants (Q2_K–Q6_K); none tested on the NPU yet.

8 accepted formats still untested

NEXT

Re-run the screening for each format with the placement check, then the episodes for the best ones.

WE'D MEASURE

prefill and decode tok/s · NPU memory · rank-1 accuracy · full placement on HTP0

03

Spend the idle NPU time

TODAY

The NPU works ~1.5–1.7 s of each 30 s tick, so most of its time is free.

When belief is weak, the model is right 0.88 of the time against belief's 0.18.

1.5–1.7 s of NPU work per 30 s tick

NEXT

When belief is unsure, ask short targeted questions: a yes/no check per candidate cause, head-to-head comparisons, several samples. Short answers keep decode cheap.

WE'D MEASURE

accuracy gained per extra NPU second · deadline misses · chip temperature